

SISTEMA DE GESTIÓN DE RESTAURANTE

APLICACIÓN DE POO, SOLID Y PATRONES DE DISEÑO

JAASIEL ALDANA
IGNACIO ANTÍAS
FRANCO DIAZ
JUAN AGUILAR
IGNACIO ITURRA

DEPARTAMENTO DE CIENCIAS DE LA INGENIERÍA
INGENIERÍA CIVIL EN INFORMÁTICA

Campus Osorno

Av. Fuchslocher 1305
Teléfono +56 64 2333 000
Fax +56 64 2333 774
Osorno, Chile

Campus Puerto Montt

Camino a Chingihue Km 6
Teléfono +56 65 2322 536
Puerto Montt, Chile

Sede Santiago

República 517
Barrio Universitario
Teléfono +56 02 2675 3057
Santiago, Chile

Sede Chiloé

Ubaldo Mansilla Barrientos 131
Teléfono 56 65 2322 409
Castro, Chile
Eleuterio Ramírez 348
Teléfono +56 65 2322 476
Ancud, Chile

TABLA DE CONTENIDO

- 1 Introducción
- 2 Objetivos
- 3 Funcionalidades
- 4 Diagrama UML
- 5 Uso de Patrones de Diseño
- 6 SOLID
- 7 Aplicación de SOLID
 - SRP
 - OCP
 - LSP
 - ISP
 - DIP
- 8 Conclusión

SECCIÓN SIGUIENTE

- 1 **Introducción**
- 2 Objetivos
- 3 Funcionalidades
- 4 Diagrama UML
- 5 Uso de Patrones de Diseño
- 6 SOLID
- 7 Aplicación de SOLID
 - SRP
 - OCP
 - LSP
 - ISP
 - DIP
- 8 Conclusión

INTRODUCCIÓN

► **¿Qué es el sistema?**

Sistema desarrollado para gestionar las operaciones principales de un restaurante mediante programación orientada a objetos.

► **¿Qué permite realizar?**

Permite administrar productos, pedidos, clientes y procesos internos del restaurante.

► **¿Qué problema resuelve?**

Ayuda a organizar la información y automatizar procesos evitando errores manuales.

SECCIÓN SIGUIENTE

- 1 Introducción
- 2 Objetivos**
- 3 Funcionalidades
- 4 Diagrama UML
- 5 Uso de Patrones de Diseño
- 6 SOLID
- 7 Aplicación de SOLID
 - SRP
 - OCP
 - LSP
 - ISP
 - DIP
- 8 Conclusión

OBJETIVOS

- ▶ Diseñar un sistema utilizando clases y objetos.
- ▶ Implementar funcionalidades CRUD: crear, modificar, eliminar y visualizar información.
- ▶ Aplicar principios SOLID.
- ▶ Implementar un patrón estructural relacionado al problema.
- ▶ Crear un código ordenado y mantenible.

SECCIÓN SIGUIENTE

- 1 Introducción
- 2 Objetivos
- 3 Funcionalidades**
- 4 Diagrama UML
- 5 Uso de Patrones de Diseño
- 6 SOLID
- 7 Aplicación de SOLID
 - SRP
 - OCP
 - LSP
 - ISP
 - DIP
- 8 Conclusión

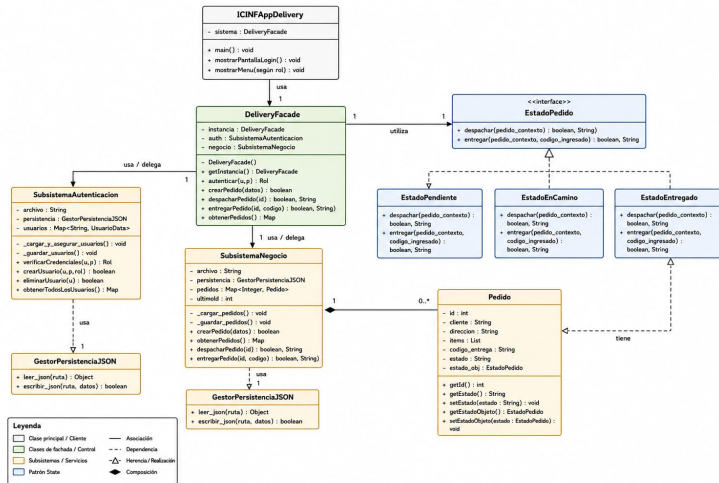
FUNCIONALIDADES DEL SISTEMA

- 1 Crear productos.
- 2 Modificar productos.
- 3 Eliminar productos.
- 4 Visualizar menú.
- 5 Registrar clientes.
- 6 Crear pedidos.
- 7 Modificar pedidos.
- 8 Eliminar pedidos.
- 9 Calcular total.
- 10 Mostrar información del sistema.

SECCIÓN SIGUIENTE

- 1 Introducción
- 2 Objetivos
- 3 Funcionalidades
- 4 Diagrama UML**
- 5 Uso de Patrones de Diseño
- 6 SOLID
- 7 Aplicación de SOLID
 - SRP
 - OCP
 - LSP
 - ISP
 - DIP
- 8 Conclusión

DIAGRAMA UML



SECCIÓN SIGUIENTE

- 1 Introducción
- 2 Objetivos
- 3 Funcionalidades
- 4 Diagrama UML
- 5 Uso de Patrones de Diseño**
- 6 SOLID
- 7 Aplicación de SOLID
 - SRP
 - OCP
 - LSP
 - ISP
 - DIP
- 8 Conclusión

PATRÓN CREACIONAL: SINGLETON

PATRÓN UTILIZADO: SINGLETON

Clase implementada: **DeliveryFacade**

¿Qué hace?

- ▶ Garantiza que exista una única instancia de la clase durante toda la ejecución del programa.
- ▶ Proporciona un punto de acceso global a dicha instancia.
- ▶ Toda la aplicación utiliza la misma fachada para comunicarse con los distintos subsistemas del restaurante.

Ventajas

- ▶ Existe una única instancia de la fachada.
- ▶ Evita inconsistencias entre múltiples objetos.
- ▶ Reduce el consumo de memoria.
- ▶ Facilita el mantenimiento del código.

PATRÓN ESTRUCTURAL: FACADE

PATRÓN UTILIZADO: FACADE (FACHADA)

Clase implementada: **DeliveryFacade**

¿Cómo funciona?

- ▶ La interfaz gráfica nunca accede directamente a los subsistemas internos.
- ▶ La clase **DeliveryFacade** actúa como intermediario entre la interfaz y:
 - ▶ SubsistemaAutenticacion
 - ▶ SubsistemaNegocio
- ▶ Centraliza todas las operaciones del sistema mediante una interfaz simple.

Ventajas

- ▶ Reduce el acoplamiento entre clases.
- ▶ Simplifica la comunicación entre los componentes.
- ▶ Oculta la complejidad interna del sistema.
- ▶ Facilita futuras modificaciones y el mantenimiento.

PATRÓN DE COMPORTAMIENTO: STATE

PATRÓN UTILIZADO: STATE

Clase implementada: **EstadoPedido**

¿Cómo funciona?

- ▶ Cada pedido mantiene un estado interno.
- ▶ Los estados posibles son:
 - ▶ Pendiente
 - ▶ En camino
 - ▶ Entregado
- ▶ El comportamiento del pedido cambia automáticamente según el estado en que se encuentre.
- ▶ Es posible agregar nuevos estados sin modificar la lógica principal del sistema.

Ventajas

- ▶ Organiza el flujo de estados del pedido.
- ▶ Separa responsabilidades entre las clases.

SECCIÓN SIGUIENTE

- 1 Introducción
- 2 Objetivos
- 3 Funcionalidades
- 4 Diagrama UML
- 5 Uso de Patrones de Diseño
- 6 SOLID**
- 7 Aplicación de SOLID
 - SRP
 - OCP
 - LSP
 - ISP
 - DIP
- 8 Conclusión

SOLID

- ▶ **S - Responsabilidad única**

Cada clase tiene una función específica.

- ▶ **O - Abierto/Cerrado**

Permite agregar nuevas funcionalidades.

- ▶ **L - Sustitución**

Las clases mantienen comportamientos compatibles.

- ▶ **I - Segregación**

Interfaces simples y específicas.

- ▶ **D - Inversión**

Las clases dependen de abstracciones.

SECCIÓN SIGUIENTE

- 1 Introducción
- 2 Objetivos
- 3 Funcionalidades
- 4 Diagrama UML
- 5 Uso de Patrones de Diseño
- 6 SOLID
- 7 Aplicación de SOLID**
 - SRP
 - OCP
 - LSP
 - ISP
 - DIP
- 8 Conclusión

APLICACIÓN DE SOLID (SRP)

- ▶ El SRP (Principio de Responsabilidad Única) es usado en las siguientes clases:
 - 1 **SubsistemaAutenticación:** Se encarga únicamente de la lectura, verificación y operaciones CRUD de las credenciales de usuarios.
 - 2 **SubsistemaNegocio:** Maneja exclusivamente el carro de compras, los pedidos y el catálogo de restaurantes.
 - 3 **ICINFAppDelivery:** Es meramente la interfaz gráfica.

APLICACIÓN DE SOLID (OCP)

- ▶ El OCP (Principio Abierto/Cerrado) es usado en la siguientes clase:
 - 1 **mostrarSistemaPestanas:** La GUI es capaz de extender los modulos visuales en la interfaz de Log in sin afectar el funcionamiento correcto del sistema.

APLICACIÓN DE SOLID (LSP)

- ▶ El LSP (Liskov Substitution Principle) es usado en la siguientes clase:
 - 1 **EstadoPedido**: El sistema a la hora de querer entregar un pedido es capaz de diferenciar los estados del mismo.

APLICACIÓN DE SOLID (ISP)

- ▶ El ISP (Principio de Segregación de la Interfaz) es usado en el patrón.
 - 1 **Patrón Facade:** Los usuarios no tiene la capacidad de visualizar la interfaz que tiene el Admin o el empleado ya que no necesitan ver dicha información.

APLICACIÓN DE SOLID (DIP)

- ▶ El DIP (Principio de Inversión de Dependencias) es usado de la siguiente manera.
 - 1 La clase de alto nivel **ICINFAppDelivery** no depende de los subsistemas de bajo nivel, como lo son **SubsistemaAutenticacion** o **SubsistemaNegocio**. Depende de una capa intermedia que sería **DeliveryFacade**.

SECCIÓN SIGUIENTE

- 1 Introducción
- 2 Objetivos
- 3 Funcionalidades
- 4 Diagrama UML
- 5 Uso de Patrones de Diseño
- 6 SOLID
- 7 Aplicación de SOLID
 - SRP
 - OCP
 - LSP
 - ISP
 - DIP
- 8 Conclusión**

CONCLUSIÓN

- ▶ El desarrollo permitió aplicar conceptos avanzados de programación orientada a objetos.
- ▶ El uso de UML permitió representar la estructura del sistema.
- ▶ Los patrones de diseño permiten crear software más organizado, flexible y escalable.